Smart Library Management System

A simple **menu-driven Library Management System** developed using Python. It manages books, members, issue/return operations, searching and library summary

Features

- Add and list books
- Search books by title, author or category
- Add and list members
- Issue available books
- Return issued books
- Automatic due-date calculation
- Library summary
- JSON-based data storage
- Input validation and error handling
- Unit testing

Project Structure

```
Smart_Library_Management_System/
│
├── main.py
├── models.py
├── library.py
├── storage.py
├── validators.py
├── reports.py
├── library_data.json
├── README.md
│
└── tests/
    └── test_library.py
```

File Description

- **main.py** – Main menu and user interaction
- **models.py** – Book and Member classes
- **library.py** – Main library operations
- **storage.py** – JSON data storage
- **validators.py** – Input validation
- **reports.py** – Books, members and summary reports
- **library_data.json** – Stores library records
- **test_library.py** – Unit tests

Requirements

- Python 3.9 or later
- No external packages required

How to Run

Open the project folder in terminal and run:

python main.py

Main Menu

1. Add Book
2. List Books
3. Search Books
4. Add Member
5. List Members
6. Issue Book
7. Return Book
8. Library Summary
9. Exit

Example

Enter Book ID: B101
Enter Title: Python Basics
Enter Author: John
Enter Category: Programming

Book added successfully.

For issuing a book:

Enter Book ID: B101
Enter Member ID: M101
Enter loan period in days: 7

Book issued successfully.

The due date is calculated automatically.

Testing

Run the unit tests using:

python -m unittest discover -s tests -v

The tests cover adding books, adding members, issuing books, returning books and preventing duplicate issues.

Python Concepts Used

- Variables and data types
- Lists and dictionaries
- Functions
- Loops and conditions
- Modules
- Object-Oriented Programming
- Exception handling
- JSON file handling
- Date calculation
- Unit testing

Future Enhancements

- Tkinter GUI
- SQLite database
- Login/authentication

- Fine calculation
- Book reservation
- CSV/PDF reports
- Web application

Conclusion

The **Smart Library Management System** demonstrates how Python concepts can be used to create a practical library application with modular programming, OOP, data storage, validation and testing.